

Introduction

Game-theory problems have a reputation for scaring people off before they even read the constraints. The moment a problem says "two players take turns" and "assume optimal play," a lot of engineers mentally check out, expecting minimax trees, alpha-beta pruning, or something that belongs in a chess engine rather than a coding interview.

Stone Game IV (LeetCode 1510) is the perfect problem to break that instinct. It looks like it needs deep lookahead across every possible sequence of moves. It doesn't. Once you see the shape of the problem, it collapses into a single boolean fact per state, computed with a plain list and a loop.

This matters beyond the problem itself. Turn-based win/lose games show up constantly in interviews — Nim variants, coin games, subtraction games — and they almost all reduce to the same idea: a state is winning if you can move to *any* state where your opponent loses. Learn this pattern once here, and you'll recognize it instantly the next time it's wearing a different costume.

Problem Overview

Alice and Bob play a game with a pile of `n` stones. Alice always moves first. On each turn, the player to move must remove a **perfect square** number of stones from the pile — 1, 4, 9, 16, and so on — as long as that many stones are actually available.

If it's your turn and there is no perfect square you can legally remove (which only happens when the pile is empty), you lose.

Given `n`, the task is to determine whether Alice wins, assuming both players always play optimally.

Constraints put `n` up to 100,000, which immediately rules out brute-force tree exploration and pushes us toward a solution that reuses work instead of recomputing it.

Example

Example 1: `n = 1` Alice removes 1 stone ($1^2 = 1$). The pile is empty. Bob has no move and loses.

Output: Alice wins.

Example 2: `n = 2` Alice's only legal move is to remove 1 stone, leaving 1 stone for Bob. Bob then removes that 1 stone, and Alice is left with nothing. **Output: Alice loses.** Two stones is a trap —

there's no way around it.

Example 3: $n = 4$ 4 is itself a perfect square (2^2). Alice removes all 4 stones in one move, leaving Bob with an empty pile. **Output: Alice wins.**

Intuition

Start by playing small cases by hand, because the pattern reveals itself faster than any formula:

- **$n = 0$:** The player to move has no stones and no legal move. They lose automatically.
- **$n = 1$:** Take 1, leave 0 for the opponent. Since the opponent now faces a losing position, you win.
- **$n = 2$:** Your only move takes 1, leaving 1 stone — a winning position for whoever faces it. So you lose.
- **$n = 3$:** Take 1, leave 2. We just showed 2 is a losing position for whoever's turn it is. So you win.
- **$n = 4$:** Take 4 outright. You win immediately.

Notice the pattern forming: every result depends entirely on smaller results we've already worked out. That's the signal that we're looking at an **optimal substructure** problem — the kind dynamic programming is built for.

Here's the rule, stated precisely: **a pile size is a winning position if there exists at least one perfect square you can remove that leaves your opponent facing a losing position.** You don't need every move to work in your favor. You only need one. Call this the "loser flip" — your entire job each turn is hunting for a move that flips your opponent into the losing seat.

This means we only need to track one fact per pile size: true if the player about to move wins, false if they lose. Everything else falls out of that single bit, checked against smaller pile sizes that are already solved.

Brute Force Solution

Idea: Recursively try every legal square move from the current pile size, and recurse into whichever pile size remains. If *any* branch leads to a win, the current state is a win.

Algorithm: 1. Base case: `n == 0` means the current player loses. 2. Try every square `j*j <= n`. 3. Recurse on `n - j*j`. If that recursive call returns `False` (opponent loses), the current state is `True`. 4. If no square move produces a losing state for the opponent, the current state is `False`.

Advantages: Trivial to reason about, directly mirrors the problem statement, easy to get right on the first try.

Disadvantages: Massive redundant work. The same smaller pile size (say, 5) gets recomputed every time it's reachable from multiple larger states, and the recursion tree grows exponentially. For n up to 100,000, this version simply never finishes.

Complexity: Time is exponential in the worst case, roughly $O(2^{\sqrt{n}})$ style blowup depending on branching. Space is $O(n)$ for the call stack depth in the worst case.

Optimal Solution

The fix is exactly what you'd expect once you spot the repeated subproblems: cache the answer for every pile size the first time you compute it, then reuse it forever after.

We build a table, `dp`, where `dp[i]` is `True` if the player about to move wins when i stones remain.

- `dp[0] = False` — no stones, no move, automatic loss.
- For every i from 1 to n : try every perfect square $j*j \leq i$. If `dp[i - j*j]` is `False` for any such j , then `dp[i] = True` — you've found a move that hands your opponent a losing position. The moment you find one, stop checking; you don't need more than one.
- If no square move produces a `False` state, `dp[i]` stays `False`.

We fill this table **left to right**, from small pile sizes to large. That ordering guarantees that by the time we need `dp[i - j*j]`, it's already been solved — nothing is computed out of order, and nothing is computed twice.

i Squares tried Result leaves opponent at `dp[i]`

0	—	False
1	1	0 (False)
2	1	1 (True)
3	1	2 (False)
4	1, 4	4 $\rightarrow$ 0 (False)
5	1, 4	4 (True), 1 (True)
6	1, 4	5 (False)
7	1, 4	6 (True), 3 (True)

A quick note on cost: at $i = 8$, j only reaches 2 (since $3*3 = 9$ overshoots). At $i = 9$, j reaches exactly 3. The inner loop only runs about $\sqrt{i}$ times, not i times — that's the entire reason this approach stays fast at $n = 100,000$.

Python Code

```
def winner_square_game(n: int) -> bool:
    """Return True if the player who moves first wins with n stones."""
    dp = [False] * (n + 1)

    for i in range(1, n + 1):
        j = 1
        while j * j <= i:
            if not dp[i - j * j]:
                dp[i] = True
                break
            j += 1

    return dp[n]
```

Code Walkthrough

- `dp = [False] * (n + 1)` — one boolean slot per pile size from `0` to `n`, all initialized to a loss since nothing has been proven yet.
- `for i in range(1, n + 1)` — solve every pile size in increasing order, so smaller answers are always ready before they're needed.
- `j = 1 / while j * j <= i` — walk through every perfect square that legally fits in the current pile size.
- `if not dp[i - j * j]` — check whether removing this square leaves the opponent in a losing state.
- `dp[i] = True; break` — the instant we find one winning move, we lock in the result and stop scanning further squares; we don't need every move to work, just one.
- `j += 1` — otherwise, try the next square.
- `return dp[n]` — the final answer for the actual starting pile size.

Dry Run

Let's trace `n = 7` using the table already built:

```
dp[0..6] = [False, True, False, True, True, False, True]
```

At `i = 7` : - `j = 1` : remove 1, leaving 6. `dp[6] = True` . No good — that hands Bob a winning position. - `j = 2` : remove 4, leaving 3. `dp[3] = True` . Still no good. - `j = 3` would need `9` , which exceeds `7` , so the loop stops.

No square move leaves Bob in a losing spot, so `dp[7] = False` . **Alice loses if the pile starts at 7 stones.**

Try it yourself for $n = 5$ before reading on: with $dp[0] = \text{False}$, $dp[1] = \text{True}$, $dp[2] = \text{False}$, $dp[3] = \text{True}$, $dp[4] = \text{True}$, check whether removing 1 (leaving 4) or removing 4 (leaving 1) hands the opponent a `False` state. Neither does — both leave `True` states — so $dp[5] = \text{False}$. Alice loses at 5 stones too.

Complexity Analysis

Time: $O(n\sqrt{n})$. For every pile size i from 1 to n , the inner loop checks roughly $\sqrt{i}$ perfect squares. Summed across all i , this comes out to $O(n\sqrt{n})$ — comfortably fast even at $n = 100,000$.

Space: $O(n)$ for the `dp` list. No recursion stack, no auxiliary structures beyond the table itself.

Both bounds are tight because every pile size must be visited exactly once (we can't skip any, since larger states depend on smaller ones), and every visit does genuine, non-repeated work proportional to $\sqrt{i}$.

Alternative Solutions

The same logic can be written as top-down recursion with memoization instead of a bottom-up loop:

```
from functools import lru_cache

def winner_square_game(n: int) -> bool:
    @lru_cache(maxsize=None)
    def wins(remaining: int) -> bool:
        if remaining == 0:
            return False
        j = 1
        while j * j <= remaining:
            if not wins(remaining - j * j):
                return True
            j += 1
        return False
    return wins(n)
```

This reads a little more naturally to some people, since it mirrors the recursive definition directly. But it comes with a real cost: recursion depth can approach n in the worst case (repeatedly removing 1 stone), which risks hitting Python's recursion limit for large inputs. The bottom-up table sidesteps this completely by construction. For this problem, prefer the loop.

Edge Cases

- **$n = 0$:** Not reachable per problem constraints ($n \geq 1$), but `dp[0] = False` correctly represents "no stones, no move."
- **$n = 1$:** Smallest real case. Alice takes the only stone and wins immediately.
- **Perfect square n (e.g., 4, 9, 16):** Always a win for the player to move — they can clear the entire pile in one move, though the DP still correctly handles cases where clearing it isn't actually the best move for larger perfect squares.
- **$n = 100,000$ (upper bound):** Verifies the solution runs in reasonable time; $O(n\sqrt{n})$ at this size is on the order of tens of millions of operations, well within limits.
- **Consecutive losing positions:** Values like 2, 5, 7 show losing positions aren't evenly spaced — don't assume a simple modular pattern exists.

Common Mistakes

1. **Always grabbing the biggest available square.** At 8 stones, taking 4 leaves 4 — a winning position for the opponent. Taking 1 and leaving 7 (a losing position) is correct. Greedily maximizing the move size is not the same as maximizing your win chance.
2. **Stopping the inner loop too early or checking only one square.** You must check every square up to $\sqrt{i}$; missing one might miss the only winning move.
3. **Filling the DP table out of order.** If you compute `dp[i]` before smaller dependent states are solved, you'll read stale or default values.
4. **Off-by-one errors in the table size.** The table needs indices `0` through `n` inclusive — a table of size `n` instead of `n + 1` will crash or silently misindex.
5. **Confusing "the player who wins" with "Alice specifically."** `dp[n]` tells you whether the player to move wins — since Alice moves first, `dp[n] = True` means Alice wins, but this framing breaks if you're not careful about whose turn it represents at recursive depth.
6. **Using unbounded recursion without memoization,** which reproduces the brute-force blowup this problem is designed to punish.

Interview Questions

- How would you modify this if players could also remove a fixed number of stones (like exactly 1) in addition to perfect squares?
- Can you extend this to return the optimal first move, not just win/lose?
- How does this generalize to other "subtract a set of allowed amounts" games?
- What changes if the losing condition were reversed (last player to move loses, a misère variant)?

- Can you prove why checking just one losing successor state is sufficient, rather than needing to check all of them?

Similar Problems

- **LeetCode 292 — Nim Game:** The simplest version of this win/lose DP pattern, solvable in $O(1)$ once you spot the modular pattern, but conceptually the same "any move to a losing state wins" logic.
- **LeetCode 877 — Stone Game:** A different stone-pile game where players pick from either end; uses interval DP rather than a single 1D state.
- **LeetCode 1140 — Stone Game II:** Introduces a variable move-size cap M , requiring a 2D DP state.
- **LeetCode 464 — Can I Win:** Same "does any move lead to an opponent loss" pattern, applied to picking numbers instead of stones.
- **LeetCode 375 — Guess Number Higher or Lower II:** A related minimax DP, though it optimizes cost rather than a binary win/lose outcome.

Key Takeaways

Stone Game IV isn't really about stones or squares — it's about recognizing a reusable shape: in any turn-based win/lose game, a state is winning if and only if some legal move lands the opponent in a losing state. Once you define that recurrence, an exponential recursive tree collapses into a linear table where each state is computed exactly once. The $\sqrt{n}$ cost per state, rather than a full scan, is what makes this scale to 100,000 stones without breaking a sweat.

Watch the Video

Prefer to see this built step by step, dry-run included, with two live quiz rounds to test yourself before the answer is revealed? Watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series that takes LeetCode problems that look like they need a specialized background — game theory, advanced math, obscure data structures — and reduces them to one idea you'll actually remember and reuse. New daily problems drop regularly, each one built the same way: intuition first, code second.

Call To Action

If this walkthrough made Stone Game IV click, the video version has the full dry run and two pause-and-guess quiz rounds — give it a watch and a like on YouTube. Subscribe to the channel for daily LeetCode breakdowns, and subscribe to the Substack for deeper dives that don't fit in a video. Drop your $n = 5$ guess (or any questions) in the comments, and share this with anyone prepping for coding interviews.

Quick Review

What pattern signals: game where players remove items, last-move-wins?

Game theory DP — think 'can current player force a win from this state' (win/lose states).

Time complexity of Stone Game IV solution?

$O(n * \sqrt{n})$ — for each of n states, try all square roots up to $\sqrt{n}$.

Space complexity of the DP array approach?

$O(n)$ for the boolean dp array tracking win/lose per remaining stone count.

Trickiest bug when coding the win condition?

Forgetting $dp[0] = \text{false}$ (no stones left = current player already lost, previous player won).

Stone Game IV vs Nim game — key difference?

Nim has closed-form XOR trick; here moves (perfect squares) aren't uniform, so no shortcut — must brute-force DP.

Follow-up: how would you handle n up to 10^9 ?

$O(n * \sqrt{n})$ too slow; would need math/number-theory insight or memorized game-state periodicity, not plain DP.